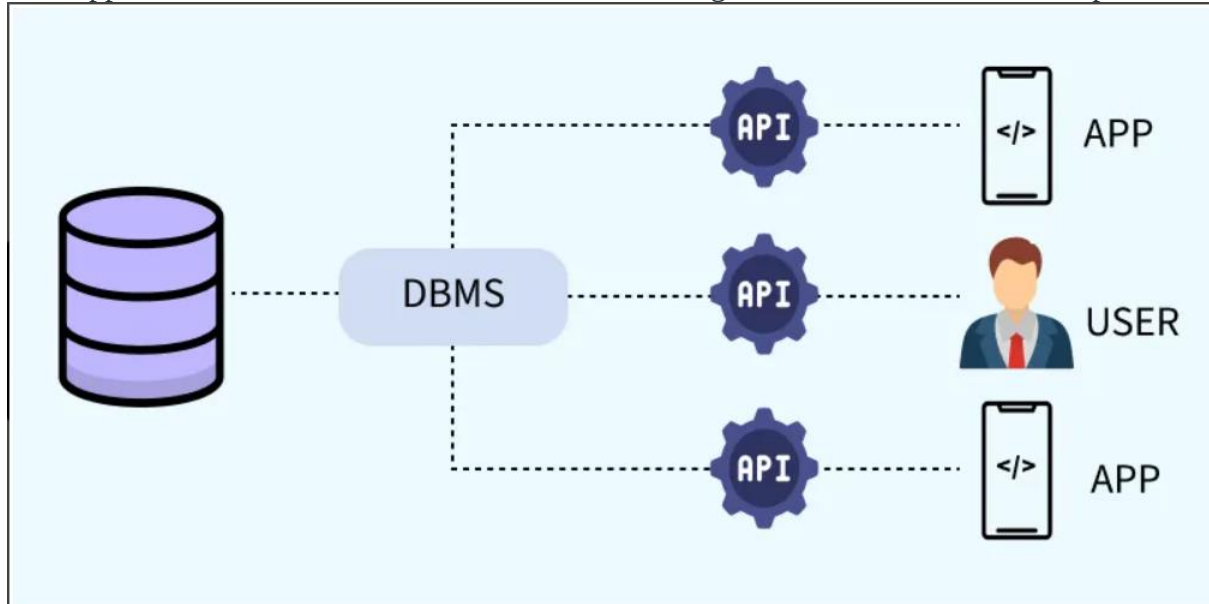


Introduction of DBMS

DBMS (Database Management System) is a software system that manages, stores, and retrieves data efficiently in a structured format.

- It allows users to create, update, and query databases efficiently.
- Ensures data integrity, consistency, and security across multiple users and applications.
- Reduces data redundancy and inconsistency through centralized control.
- Supports concurrent data access, transaction management, and automatic backups



DBMS acts as a bridge between a central database and multiple clients-including apps and users. It uses **APIs** to handle data requests, enabling apps and users to interact with the database securely and efficiently without directly accessing the data.

Problems with Traditional File-Based Systems

Before the introduction of modern DBMS, data was managed using basic file systems on hard drives. While this approach allowed users to store, retrieve and update files as needed, it came with numerous challenges

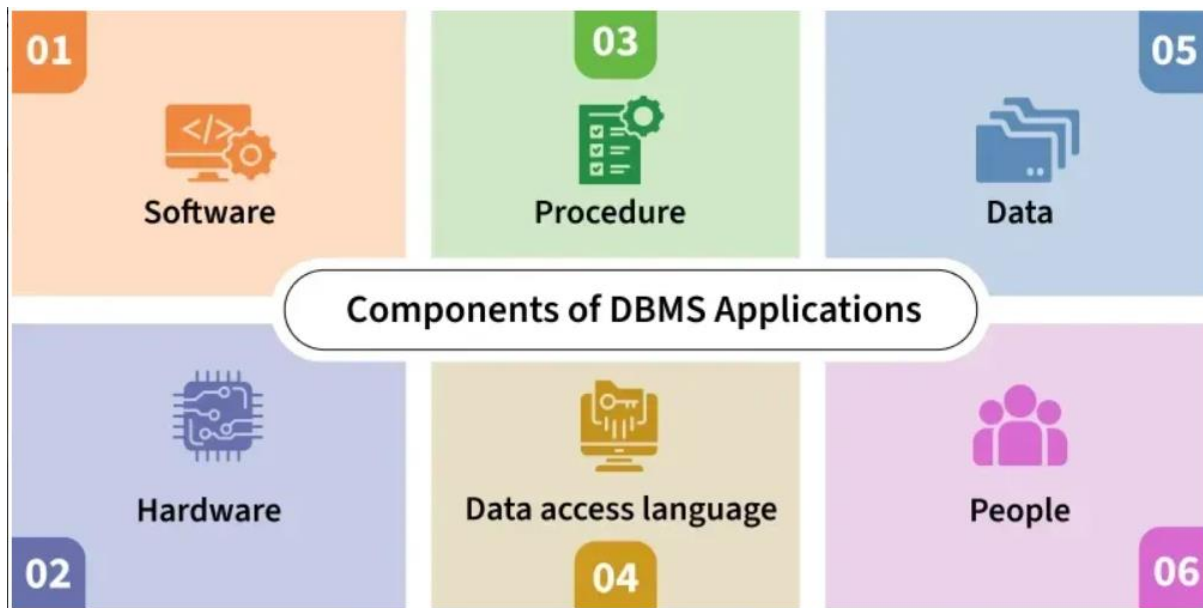
- **Data Redundancy:** Duplicate entries across files
- **Inconsistency:** Conflicting or outdated information
- **Difficult Access:** Manual file search required
- **Poor Security:** No control over data access
- **Single-User Access:** No support for collaboration
- **No Backup/Recovery:** Data loss was often permanent

A university file-based system storing data in separate files (e.g., Academics, Results, Hostels) often faced these problems.

Components of DBMS Applications

Any DBMS based applications is made up of six key components that work together to handle data effectively.

Components of DBMS Applications



1. Hardware

- Physical devices like servers, disks, input-output devices (keyboard, monitor, printer).
- Stores and processes data; interfaces between real-world inputs and digital systems.
- Examples: Personal computer hard disk, RAM, network devices used for DBMS operations.

2. Software

- Actual DBMS software like MySQL, Oracle, PostgreSQL.
- Includes the database engine, OS, network software, and application tools.
- Translates database access languages into operations.

3. Data

- Raw facts stored in structured or unstructured formats.
- **Operational Data**: Actual user data (e.g., name, age).
- **Metadata**: Data about data (e.g., storage time, size, data type).
- Core reason DBMS exists—to manage and store data efficiently.

4. Procedures

- Instructions and rules for using DBMS effectively.
- Covers setup, login/logout, data validation, backup, access control, and report generation.
- Helps ensure consistent and secure use of the system.

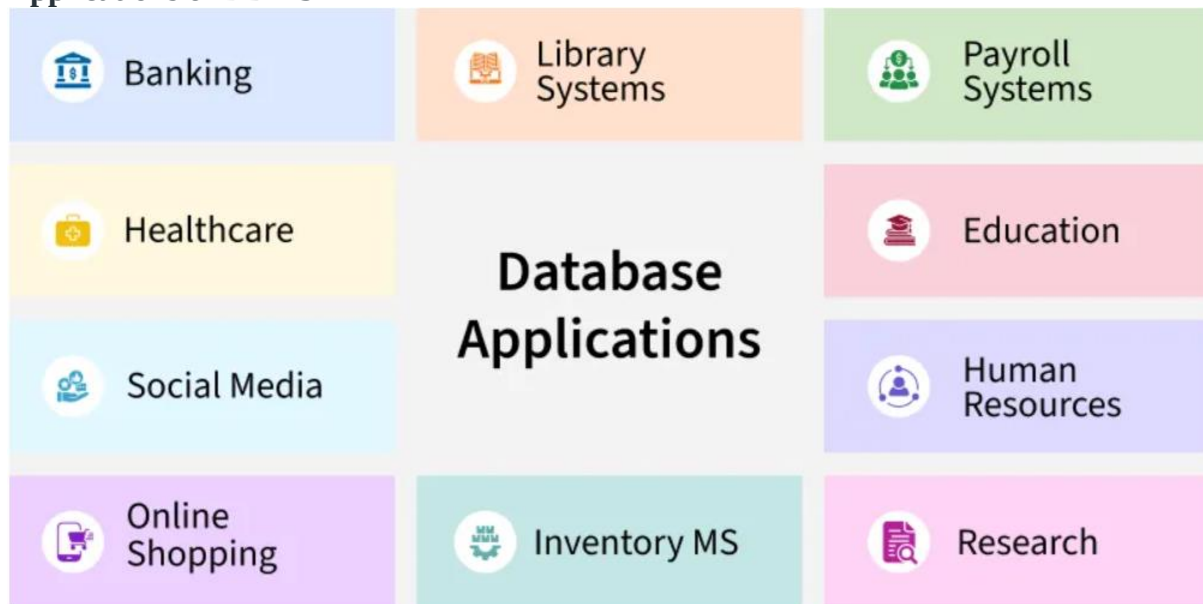
5. Database Access Language

- Used to interact with the database (create, read, update, delete data).
- Examples: SQL, MyAccess, Oracle PL/SQL.
- **DDL (Data Definition Language)** – CREATE, ALTER, DROP
- **DML (Data Manipulation Language)** – INSERT, UPDATE, DELETE

6. People

- Users interacting with DBMS at different levels:
- **Database Administrators (DBA)** – Manage security, performance, user access.
- **Developers** – Build applications using the database.
- **End Users** – Use applications to access the database (e.g., students, employees).

Applications of DBMS



- **Banking:** Manages accounts and transactions.
- **E-commerce:** Tracks products, orders, and customers.
- **Healthcare:** Stores patient records and diagnoses.
- **Education:** Handles student grades and schedules.
- **Social Media:** Manages user profiles and interactions.
- **Data Science:** Supports analytics and predictions.

Types of DBMS

There are several types of Database Management Systems (DBMS), each tailored to different data structures, scalability requirements and application needs. The most common types are as follows:

1. Relational Database Management System (RDBMS)

- It organizes data into tables (relations) composed of rows and columns.
- Uses primary keys to uniquely identify rows and foreign keys to establish relationships between tables.
- Queries are written in **SQL (Structured Query Language)**, which allows for efficient data manipulation and retrieval.

Examples: MySQL, Oracle, Microsoft SQL Server and PostgreSQL.

2. NoSQL DBMS

- They are designed to handle large-scale data and provide high performance for scenarios where relational models might be restrictive.
- They store data in various non-relational formats, such as key-value pairs, documents, graphs or columns.
- These flexible data models enable rapid scaling and are well-suited for unstructured or semi-structured data.

Examples: MongoDB, Cassandra, DynamoDB and Redis.

3. Object-Oriented DBMS (OODBMS)

- It integrates object-oriented programming concepts into the database environment, allowing data to be stored as objects.
- Supports complex data types and relationships, making it ideal for applications requiring advanced data modeling and real-world simulations.

Examples: ObjectDB, db4o.

4. Hierarchical Database

- Organizes data in a tree-like structure, where each record (node) has a single parent and have multiple children.
- This model is similar to a file system with folders and subfolders.
- It is efficient for storing data with a clear hierarchy, such as organizational charts or file directories.
- Navigation is fast and predictable due to the fixed structure.
- It lacks flexibility and difficult to restructure or handle complex many-to-many relationships.

Example: IBM Information Management System (IMS).

5. Network Database

- It uses a graph-like model to allow more complex relationships between entities.
- Unlike the hierarchical model, it permits each child to have multiple parents, enabling many-to-many relationships.
- Data is represented using records and sets, where sets define the relationships.
- It is more flexible than the hierarchical model and better suited for applications with complex data linkages.

Example: Integrated Data Store (IDS), TurboIMAGE.

6. Cloud-Based Database

- They are hosted on cloud computing platforms like AWS, Azure or Google Cloud.
- They offer on-demand scalability, high availability, automatic backups and remote accessibility.
- These databases can be relational (SQL) or non-relational (NoSQL) and are maintained by cloud service providers, reducing administrative overhead.
- They support modern application requirements, including distributed access and real-time analytics.

Example: Amazon RDS (for SQL), MongoDB Atlas (for NoSQL), Google BigQuery.

DBMS Architecture 1-level, 2-Level, 3-Level

- A DBMS architecture defines how users interact with the database to read, write, or update information. A well-designed architecture and schema (a blueprint detailing tables, fields and relationships) ensure data consistency, improve performance and keep data secure.

Types of DBMS Architecture

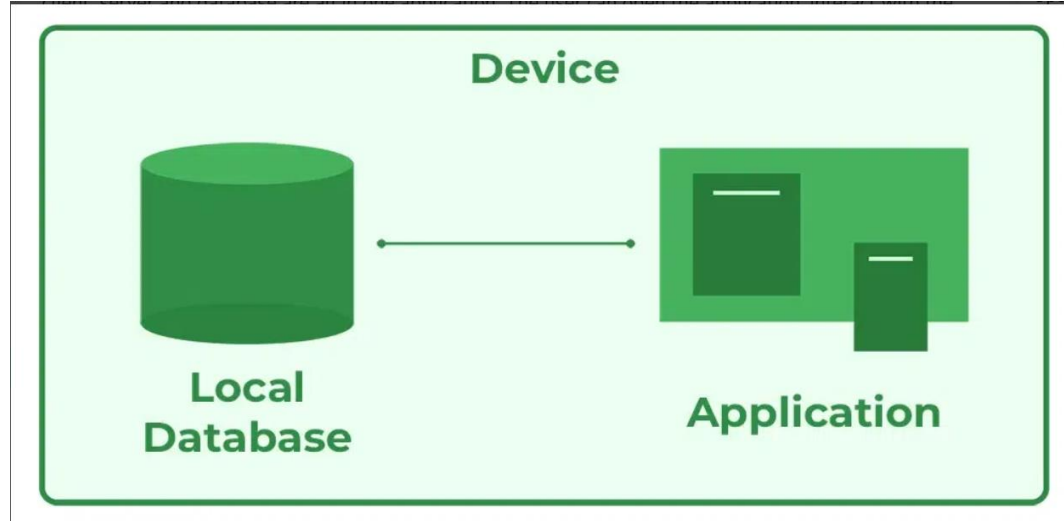
There are several types of DBMS Architecture that we use according to the usage requirements.

- 1-Tier Architecture
- 2-Tier Architecture

- 3-Tier Architecture

1-Tier Architecture

In 1-Tier Architecture, the user works directly with the database on the same system. This means the client, server and database are all in one application. The user can open the application, interact with the data and perform tasks without needing a separate server or network connection.



DBMS 1-Tier Architecture

- A common example is Microsoft Excel. Everything from the user interface to the logic and data storage happens on the same device. The user enters data, performs calculations and saves files directly on their computer.
- This setup is simple and easy to use, making it ideal for personal or standalone applications. It does not require a network or complex setup, which is why it's often used in small-scale or individual use cases.
- This architecture is simple and works well for personal, standalone applications where no external server or network connection is needed.

Advantages of 1-Tier Architecture

Below mentioned are the advantages of 1-Tier Architecture.

- **Simple Architecture:** 1-Tier Architecture is the most simple architecture to set up, as only a single machine is required to maintain it.
- **Cost-Effective:** No additional hardware is required for implementing 1-Tier Architecture, which makes it cost-effective.
- **Easy to Implement:** 1-Tier Architecture can be easily deployed and hence it is mostly used in small projects.

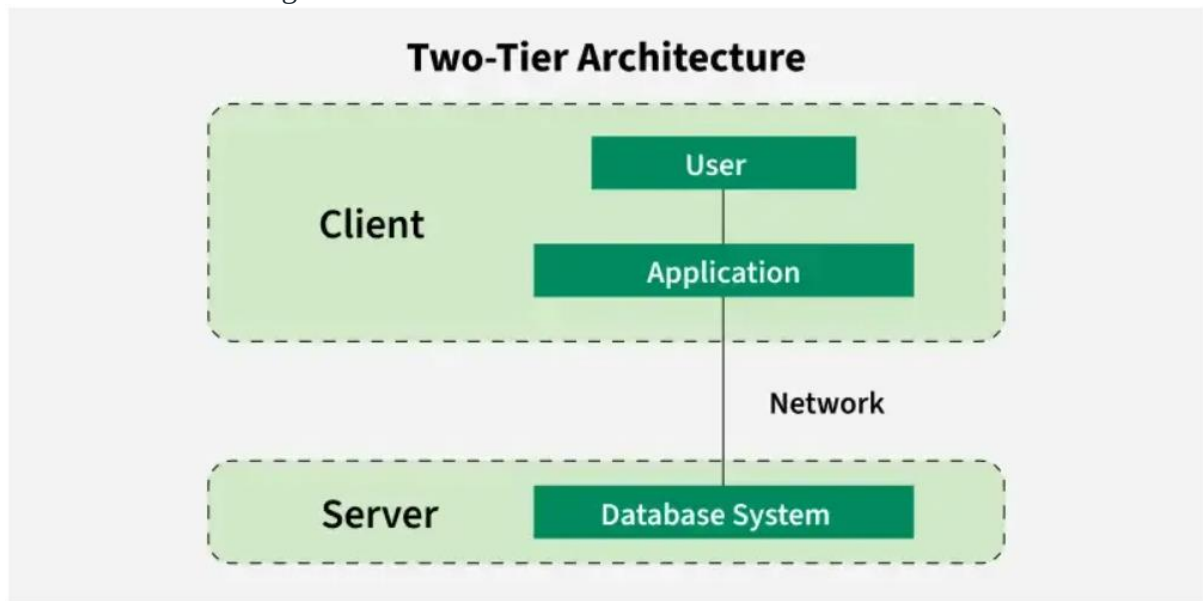
Disadvantages of 1-Tier Architecture

- **Limited to Single User:** Only one person can use the application at a time. It's not designed for multiple users or teamwork.
- **Poor Security:** Since everything is on the same machine, if someone gets access to the system, they can access both the data and the application easily.

- **No Centralized Control:** Data is stored locally, so there's no central database. This makes it hard to manage or back up data across multiple devices.
- **Hard to Share Data:** Sharing data between users is difficult because everything is stored on one computer.

2-Tier Architecture

The 2-tier architecture is similar to a basic [client-server model](#). The application at the client end directly communicates with the database on the server side. APIs like ODBC and JDBC are used for this interaction. The server side is responsible for providing query processing and transaction management functionalities.



DBMS 2-Tier Architecture

- On the client side, the user interfaces and application programs are run. The application on the client side establishes a connection with the server side to communicate with the DBMS. For Example: A Library Management System used in schools or small organizations is a classic example of two-tier architecture.
- **Client Layer (Tier 1):** This is the user interface that library staff or users interact with. For example they might use a desktop application to search for books, issue them, or check due dates.
- **Database Layer (Tier 2):** The database server stores all the library records such as book details, user information and transaction logs.
- The client layer sends a request (like searching for a book) to the database layer which processes it and sends back the result. This separation allows the client to focus on the user interface, while the server handles data storage and retrieval.

Advantages of 2-Tier Architecture

- **Easy to Access:** 2-Tier Architecture makes easy access to the database, which makes fast retrieval.
- **Scalable:** We can scale the database easily, by adding clients or upgrading hardware.
- **Low Cost:** 2-Tier Architecture is cheaper than 3-Tier Architecture and Multi-Tier Architecture.
- **Easy Deployment:** 2-Tier Architecture is easier to deploy than 3-Tier Architecture.

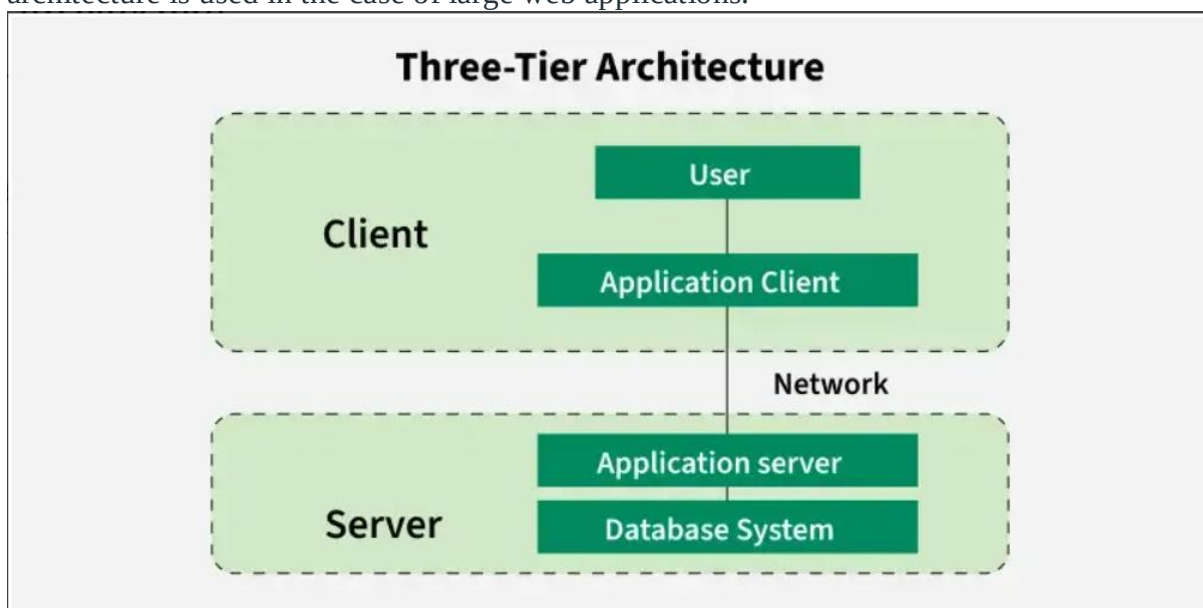
- **Simple:** 2-Tier Architecture is easily understandable as well as simple because of only two components.

Disadvantages of 2-Tier Architecture

- **Limited Scalability:** As the number of users increases, the system performance can slow down because the server gets overloaded with too many requests.
- **Security Issues:** Clients connect directly to the database, which can make the system more vulnerable to attacks or data leaks.
- **Tight Coupling:** The client and the server are closely linked. If the database changes, the client application often needs to be updated too.
- **Difficult Maintenance:** Managing updates, fixing bugs, or adding features becomes harder when the number of users or systems increases.

3-Tier Architecture

In 3-Tier Architecture, there is another layer between the client and the server. The client does not directly communicate with the server. Instead, it interacts with an application server which further communicates with the database system and then the query processing and transaction management takes place. This intermediate layer acts as a medium for the exchange of partially processed data between the server and the client. This type of architecture is used in the case of large web applications.



DBMS 3-Tier Architecture

Example: E-commerce Store

- **User:** You visit an online store, search for a product and add it to your cart.
- **Processing:** The system checks if the product is in stock, calculates the total price and applies any discounts.
- **Database:** The product details, your cart and order history are stored in the database for future reference.

Advantages of 3-Tier Architecture

- **Enhanced scalability:** Scalability is enhanced due to the distributed deployment of application servers. Now, individual connections need not be made between the client and server.
- **Data Integrity:** 3-Tier Architecture maintains Data Integrity. Since there is a middle layer between the client and the server, data corruption can be avoided/removed.
- **Security:** 3-Tier Architecture Improves Security. This type of model prevents direct interaction of the client with the server thereby reducing access to unauthorized data.

Disadvantages of 3-Tier Architecture

- **More Complex:** 3-Tier Architecture is more complex in comparison to 2-Tier Architecture. Communication Points are also doubled in 3-Tier Architecture.
- **Difficult to Interact:** It becomes difficult for this sort of interaction to take place due to the presence of middle layers.
- **Slower Response Time:** Since the request passes through an extra layer (application server), it may take more time to get a response compared to 2-Tier systems.
- **Higher Cost:** Setting up and maintaining three separate layers (client, server and database) requires more hardware, software and skilled people. This makes it more expensive.

View of Data , Data Abstraction (Instances and Schemas),

View of data in DBMS

Abstraction is one of the main features of database systems. Hiding irrelevant details from user and providing abstract view of data to users, helps in easy and efficient **user-database** interaction. In the previous tutorial, we discussed the three level of DBMS architecture, The top level of that architecture is “view level”. The view level provides the “**view of data**” to the users and hides the irrelevant details such as data relationship, database schema, constraints, security etc from the user.

To fully understand the view of data, you must have a basic knowledge of data abstraction and instance & schema.

1. Data abstraction: Database systems are made-up of complex data structures. To ease the user interaction with database, the developers hide internal irrelevant details from users. This process of hiding irrelevant details from user is called data abstraction.
2. Instance and schema: Design of a database is called the schema. Schema is of three types: Physical schema, logical schema and view schema. The data stored in database at a particular moment of time is called instance of database. Database schema defines the variable declarations in tables that belong to a particular database; the value of these variables at a moment of time is called the instance of that database.

Data Abstraction and Data Independence

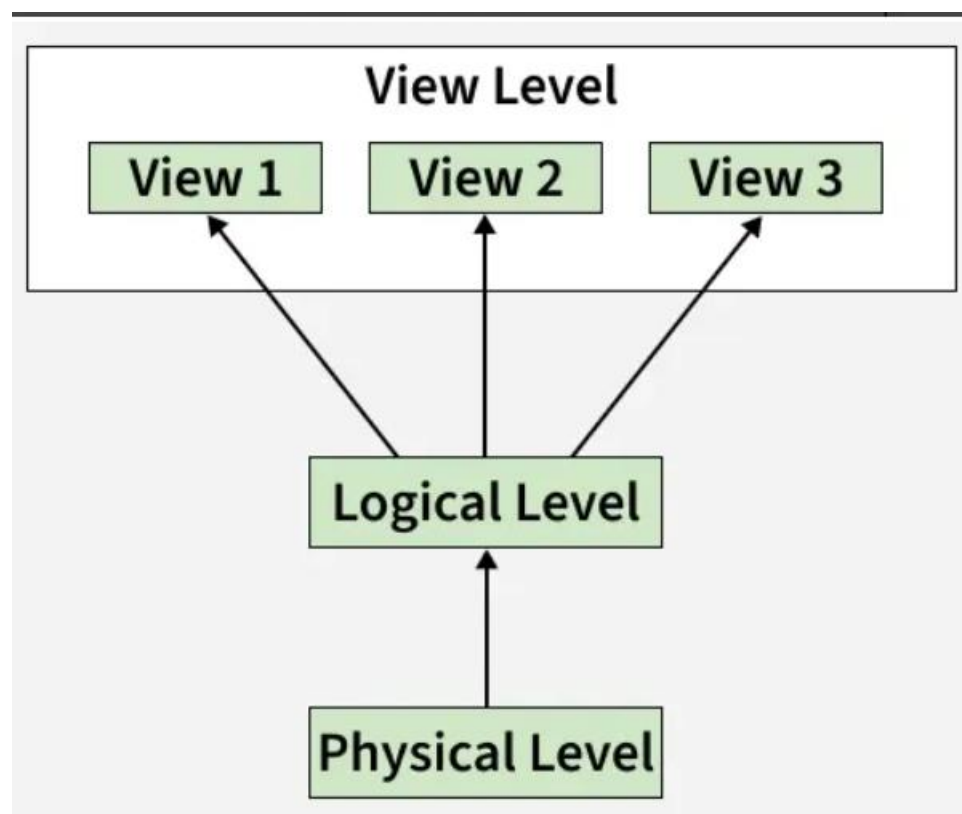
•

A Database Management System (DBMS) is designed to efficiently store, organize and retrieve large volumes of data. However, because of the complexity of databases, it is essential to simplify how users interact with the system. To achieve this,

- DBMS uses a concept called data abstraction, which hides unnecessary details and presents only relevant information to different users.
- This simplification also enables data independence, ensuring that changes in one level of the database do not affect other levels.

Data Abstraction

The main goal of data abstraction is to simplify database management and achieve data independence. It allows users to work with data without worrying about how it is stored or maintained internally, reducing complexity and improving efficiency. There are mainly three levels of data abstraction in a DBMS:



Data Abstraction

1. Physical or Internal Level
2. Logical or Conceptual Level
3. View or External Level

1. Physical or Internal Level

- This is the lowest level of abstraction that deals with how data is physically stored in memory.
- It defines file organization techniques (like hashing or B+ trees) and access methods (such as sequential or random access).
- Details such as block size, memory allocation and storage paths are managed at this level.
- These physical details are hidden from the end users.

Example: When storing employee details, the user doesn't need to know how many memory blocks are used or where the data is stored on disk.

2. Logical or Conceptual Level

- This level describes what data is stored in the database and the relationships among those data.
- It represents the entire database using simple logical structures like tables, attributes and relationships.
- The logical level provides physical data independence, meaning that changes at the physical level do not affect this level.
- Database administrators primarily work at this level to define schemas and relationships.

Example: At this level, employee details are represented as tables with fields such as Emp_ID, Emp_Name and Manager_ID.

3. View or External Level

- This is the highest level of abstraction, where users interact directly with the database.
- It provides different views of the same data for different users, ensuring security and simplicity.
- Users access data through Graphical User Interfaces (GUI) or Command-Line Interfaces (CLI) without dealing with the underlying complexity.

Example: A user may view only the Emp_Name and Department columns, while other details remain hidden.

Data Independence

Data Independence is the property that allows changes in the database schema at one level without requiring changes at the next higher level. It helps separate data from the programs that use it, thereby improving flexibility and minimizing maintenance effort.

Example of Data Independence

Consider a database storing customer information:

- **Physical level:** Stores data in files and manages memory blocks.
- **Logical level:** Represents customers as a table with attributes like Customer_ID, Name and Phone.
- **View level:** A user may see only Name and Phone fields.

Note: If the storage method changes (physical level) or a new column Email is added (logical level), the user view remains unaffected - demonstrating data independence.

There are two types of data independence:

1. Physical Data Independence
2. Logical Data Independence

1. Physical Data Independence

- It refers to the ability to change the physical storage of data without affecting the logical schema.
- Changes like altering indexes, switching file organization methods or upgrading storage devices do not affect the conceptual structure of the database.

Examples:

- Changing from sequential file organization to hashing.
- Using a new storage device or modifying access paths.

Benefit: It allows performance optimizations at the physical level without modifying higher-level structures.

2. Logical Data Independence

- It refers to the ability to modify the logical schema (tables, attributes, relationships) without affecting the user views or application programs.
- Users can continue to access data without changes in their queries even if new fields are added or relationships modified in the conceptual schema.

Examples:

- Adding or deleting attributes in a table.
- Changing the relationship between entities.

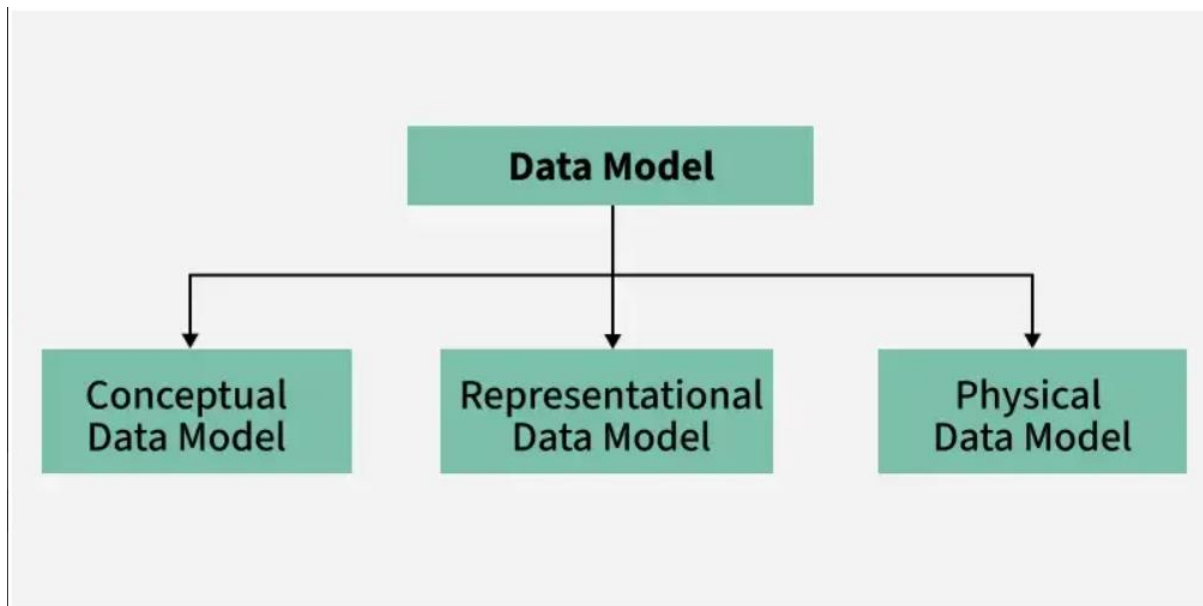
data Models , the ER Model , Relational Model, Data Models in DBMS

A Data Model in Database Management System (DBMS) is the concept of tools that are developed to summarize the description of the database. Data Models provide us with a transparent picture of data which helps us in creating an actual database. It shows us from the design of the data to its proper implementation of data.

Types of Relational Models

1. Conceptual Data Model
2. Representational Data Model
3. Physical Data Model

It is basically classified into 3 types:-



Data Models

1. Conceptual Data Model

The conceptual data model describes the database at a very high level and is useful to understand the needs or requirements of the database. It is this model, that is used in the requirement-gathering process i.e. before the Database Designers start making a particular database. One such popular model is the entity/relationship model (ER model). The E/R model specializes in entities, relationships, and even attributes that are used by database designers. In terms of this concept, a discussion can be made even with non-computer science(non-technical) users and stakeholders, and their requirements can be understood.

Entity-Relationship Model(ER Model): It is a high-level data model which is used to define the data and the relationships between them. It is basically a conceptual design of any database which is easy to design the view of data.

Components of ER Model:

1. **Entity:** An entity is referred to as a real-world object. It can be a name, place, object, class, etc. These are represented by a rectangle in an ER Diagram.
2. **Attributes:** An attribute can be defined as the description of the entity. These are represented by Ellipse in an ER Diagram. It can be Age, Roll Number, or Marks for a Student.
3. **Relationship:** Relationships are used to define relations among different entities. Diamonds and Rhombus are used to show Relationships.

Characteristics of a conceptual data model

- Offers Organization-wide coverage of the business concepts.
- This type of Data Models are designed and developed for a business audience.

- The conceptual model is developed independently of hardware specifications like data storage capacity, location or software specifications like DBMS vendor and technology. The focus is to represent data as a user will see it in the “real world.”

Conceptual data models known as Domain models create a common vocabulary for all stakeholders by establishing basic concepts and scope

2. Representational Data Model

This type of data model is used to represent only the logical part of the database and does not represent the physical structure of the database. The representational data model allows us to focus primarily, on the design part of the database. A popular representational model is a Relational model. The relational Model consists of Relational Algebra and Relational Calculus. In the Relational Model, we basically use tables to represent our data and the relationships between them. It is a theoretical concept whose practical implementation is done in Physical Data Model.

The advantage of using a Representational data model is to provide a foundation to form the base for the Physical model.

Characteristics of Representational Data Model

- Represents the logical structure of the database.
- Relational models like Relational Algebra and Relational Calculus are commonly used.
- Uses tables to represent data and relationships.
- Provides a foundation for building the physical data model.

3. Physical Data Model

The physical Data Model is used to practically implement Relational Data Model. Ultimately, all data in a database is stored physically on a secondary storage device such as discs and tapes. This is stored in the form of files, records, and certain other data structures. It has all the information on the format in which the files are present and the structure of the databases, the presence of external data structures, and their relation to each other. Here, we basically save tables in memory so they can be accessed efficiently. In order to come up with a good physical model, we have to work on the relational model in a better way. Structured Query Language (SQL) is used to practically implement Relational Algebra.

This Data Model describes HOW the system will be implemented using a specific DBMS system. This model is typically created by DBA and developers. The purpose is actual implementation of the database.

Characteristics of a physical data model:

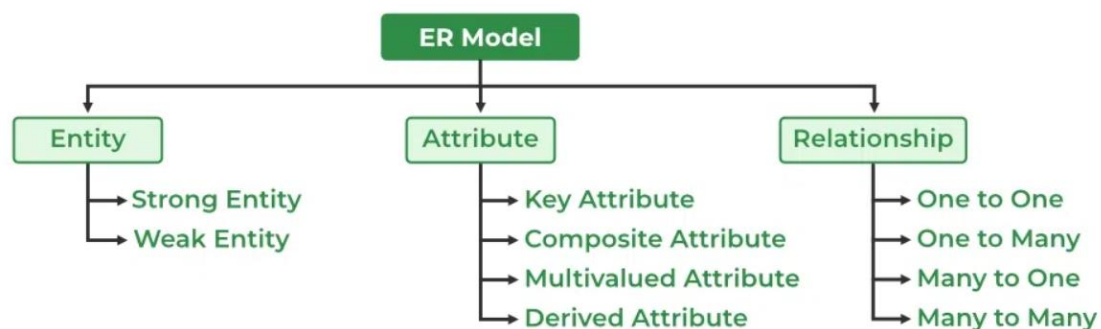
- The physical data model describes data need for a single project or application though it maybe integrated with other physical data models based on project scope.
- Data Model contains relationships between tables that which addresses cardinality and nullability of the relationships.

- Developed for a specific version of a DBMS, location, data storage or technology to be used in the project.
- Columns should have exact datatypes, lengths assigned and default values.
- Primary and Foreign keys, views, indexes, access profiles, and authorizations, etc. are defined

Introduction of ER Model

The Entity-Relationship Model (ER Model) is a conceptual model for designing a database. This model represents the logical structure of a database, including entities, their attributes, and relationships between them.

- **Entity:** An object that is stored as data. E.g: Student, Course, or Company.
- **Attribute:** Properties that describe an entity. E.g: StudentID, CourseName, or EmployeeEmail.
- **Relationship:** A connection between entities. E.g: Student enrolls in a Course.



Components of ER Diagram

ER Model in Database Design Process

We typically follow the below steps for designing a database for an application.

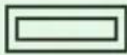
- Gather the requirements (functional and data) by asking questions to the database users.
- Create a logical or conceptual design of the database. This is where ER model plays a role. It is the most used graphical representation of the conceptual design of a database.
- After this, focus on Physical Database Design (like indexing) and external design (like views)

Use ER Diagrams In DBMS

- ER diagrams represent the E-R model in a database, making them easy to convert into relations (tables).
- These diagrams serve the purpose of real-world modeling of objects which makes them intently useful.
- Unlike technical schemas, ER diagrams require no technical knowledge of the underlying DBMS used.
- They visually model data and its relationships, making complex systems easier to understand.

Symbols Used in ER Model

ER Model is used to model the logical view of the system from a data perspective which consists of these symbols:

Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity

Symbols used in ER Diagram

Entity

An Entity represents a real-world object, concept or thing about which data is stored in a database. It act as a building block of a database. Tables in relational database represent these entities.

Example of entities:

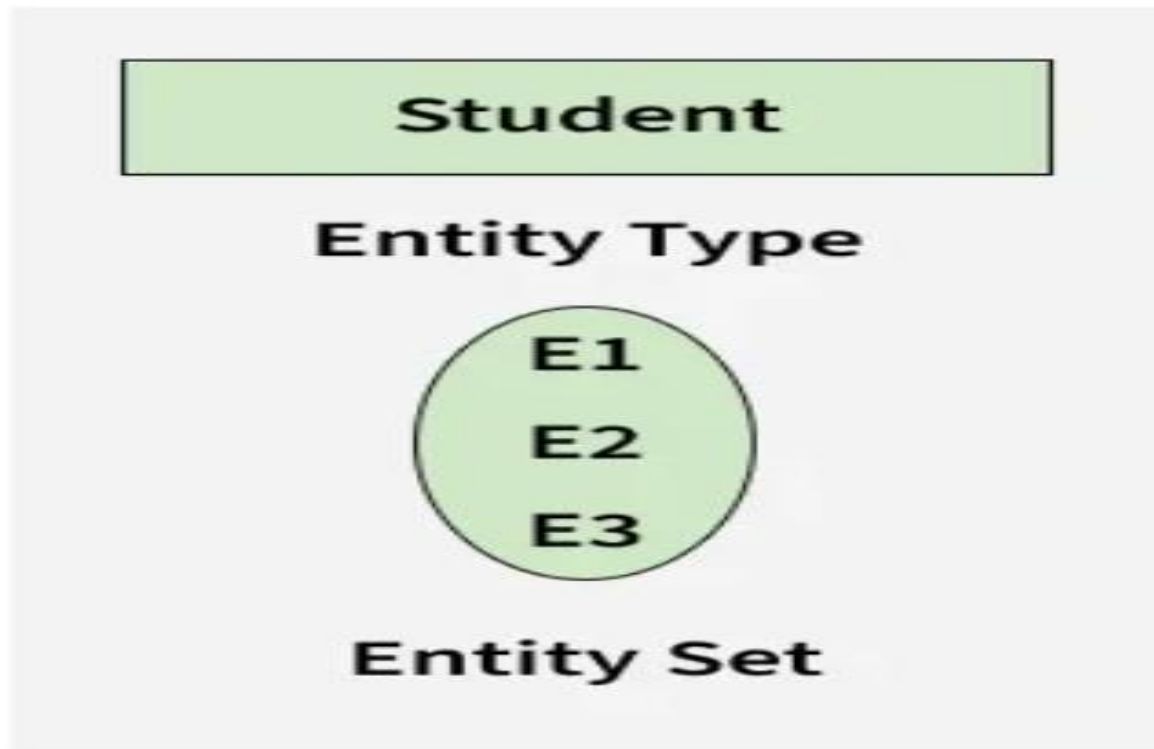
- **Real-World Objects:** Person, Car, Employee etc.
- **Concepts:** Course, Event, Reservation etc.
- **Things:** Product, Document, Device etc.

The entity type defines the structure of an entity, while individual instances of that type represent specific entities.

Entity Set

An entity refers to an individual object of an entity type, and the collection of all entities of a particular type is called an entity set. For example, E1 is an entity that belongs to the entity type "Student," and the group of all students forms the entity set.

In the ER diagram below, the entity type is represented as:



Entity Set

We can represent the entity sets in an ER Diagram but we can't represent individual entities because an entity is like a row in a table, and an ER diagram shows the structure and relationships of data, not specific data entries (like rows and columns). An ER diagram is a visual representation of the data model, not the actual data itself.

Types of Entity

There are two main types of entities:

1. Strong Entity

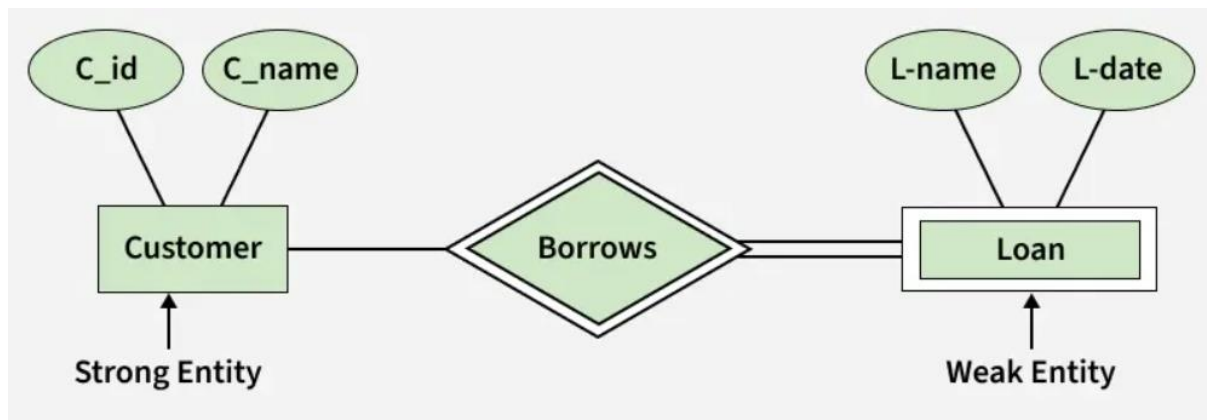
A Strong Entity is a type of entity that has a key Attribute that can uniquely identify each instance of the entity. A Strong Entity does not depend on any other Entity in the Schema for its identification. It has a primary key that ensures its uniqueness and is represented by a rectangle in an ER diagram.

2. Weak Entity

A Weak Entity cannot be uniquely identified by its own attributes alone. It depends on a strong entity to be identified. A weak entity is associated with an identifying entity (strong entity), which helps in its identification. A weak entity are represented by a double rectangle. The participation of weak entity types is always total. The relationship between the weak entity type and its identifying strong entity type is called identifying relationship and it is represented by a double diamond.

Example:

A company may store the information of dependents (Parents, Children, Spouse) of an Employee. But the dependents can't exist without the employee. So dependent will be a Weak Entity Type and Employee will be identifying entity type for dependent, which means it is Strong Entity Type.



Strong Entity and Weak Entity

Attributes in ER Model

Attribute are the properties that define the entity type. For example, for a Student entity Roll_No, Name, DOB, Age, Address, and Mobile_No are the attributes that define entity type Student. In ER diagram, the attribute is represented by an oval.



Attribute

Types of Attributes

1. Key Attribute

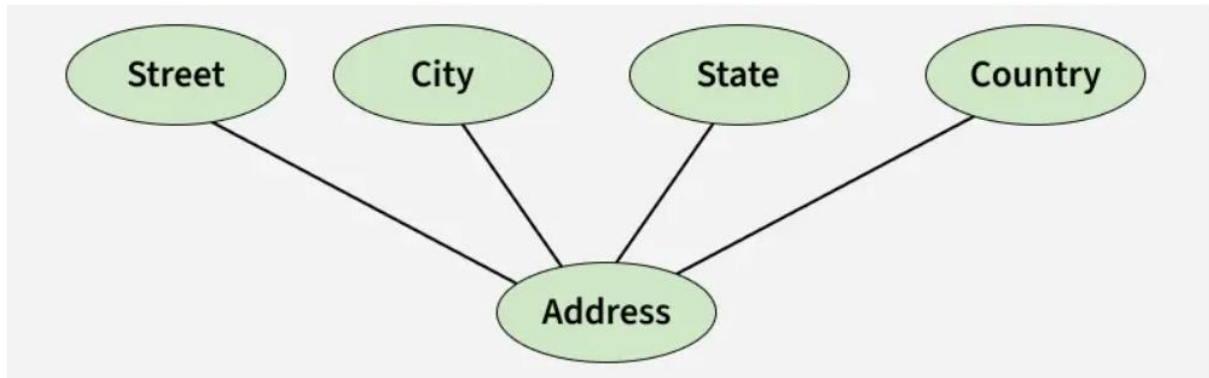
The attribute which uniquely identifies each entity in the entity set is called the key attribute. For example, Roll_No will be unique for each student. In ER diagram, the key attribute is represented by an oval with an underline.



Key Attribute

2. Composite Attribute

An attribute composed of many other attributes is called a composite attribute. For example, the Address attribute of the student Entity type consists of Street, City, State, and Country. In ER diagram, the composite attribute is represented by an oval comprising of ovals.



Composite Attribute

3. Multivalued Attribute

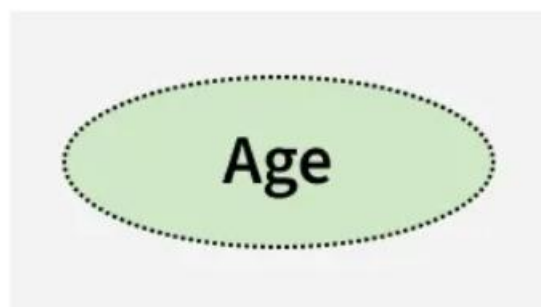
An attribute consisting of more than one value for a given entity. For example, Phone_No (can be more than one for a given student). In ER diagram, a multivalued attribute is represented by a double oval.



Multivalued Attribute

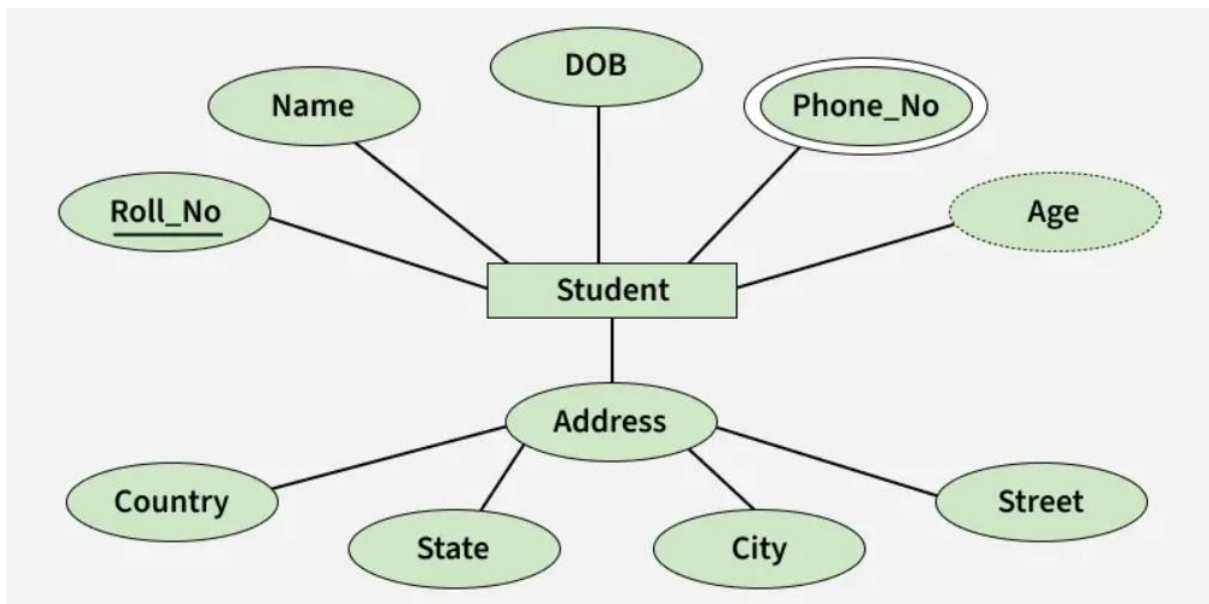
4. Derived Attribute

An attribute that can be derived from other attributes of the entity type is known as a derived attribute. e.g.; Age (can be derived from DOB). In ER diagram, the derived attribute is represented by a dashed oval.



Derived Attribute

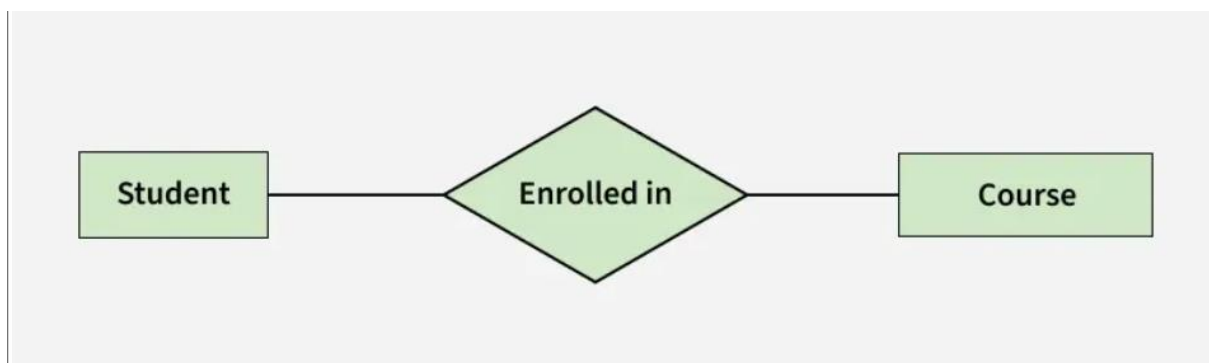
The Complete Entity Type Student with its Attributes can be represented as:



Entity and Attributes

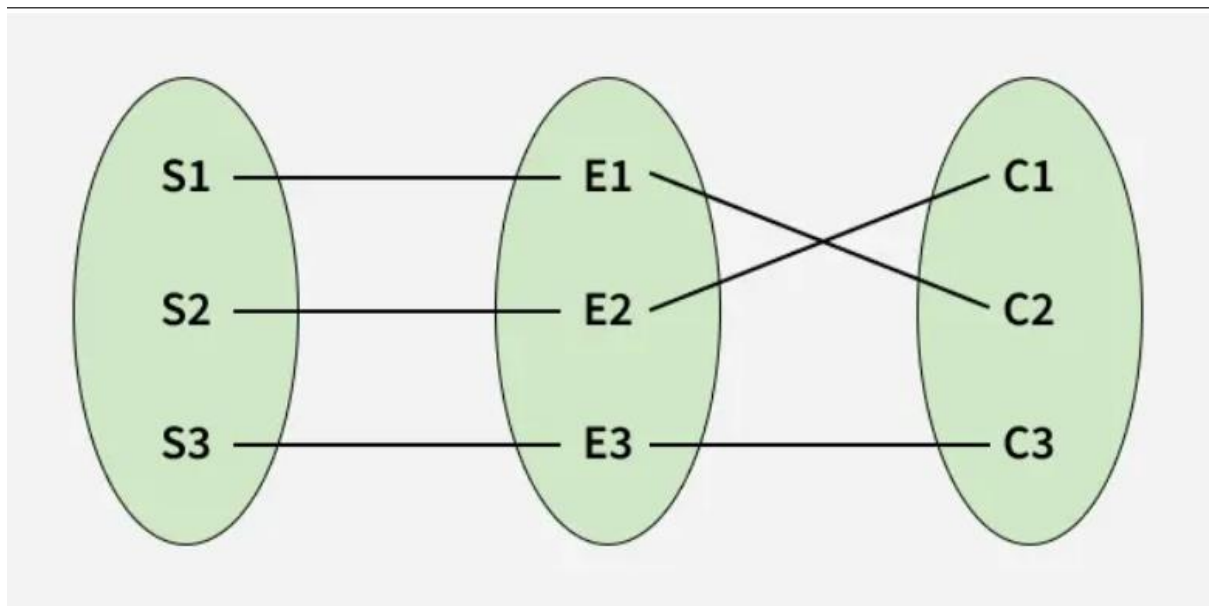
Relationship Type and Relationship Set

A Relationship Type represents the association between entity types. For example, 'Enrolled in' is a relationship type that exists between entity type Student and Course. In ER diagram, the relationship type is represented by a diamond and connecting the entities with lines.



Entity-Relationship Set

A set of relationships of the same type is known as a relationship set. The following relationship set depicts S1 as enrolled in C2, S2 as enrolled in C1, and S3 as registered in C3.

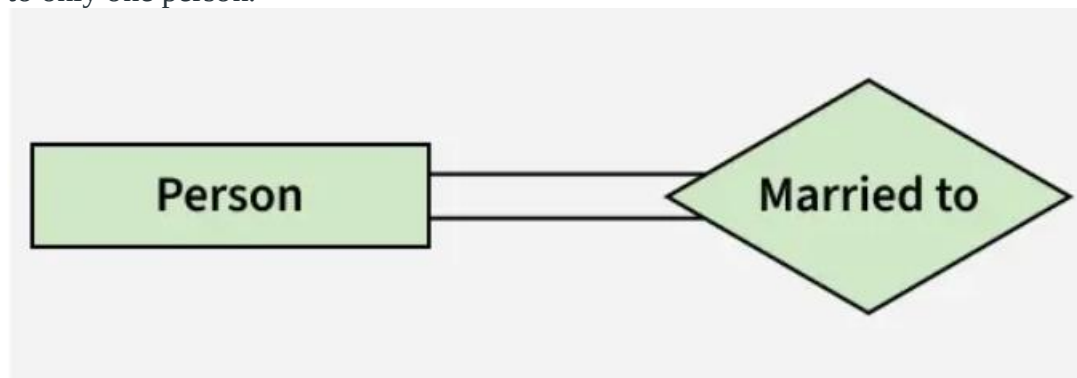


Relationship Set

Degree of a Relationship Set

The number of different entity sets participating in a relationship set is called the degree of a relationship set.

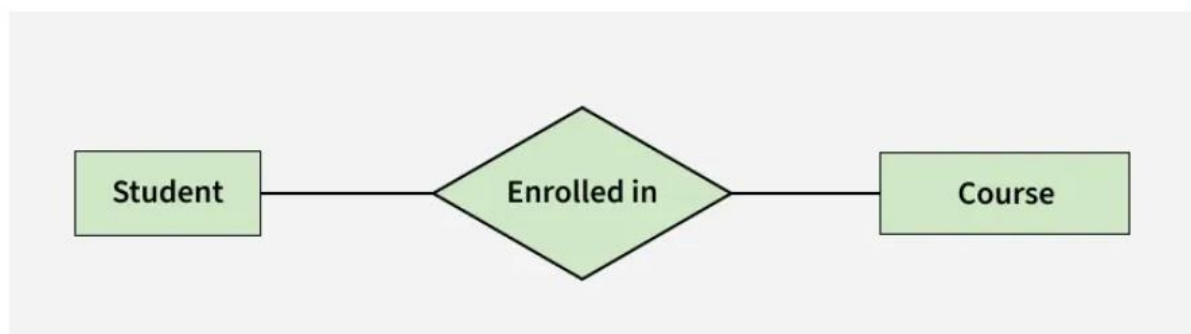
1. Unary/Recursive Relationship: When there is only ONE entity set participating in a relation, the relationship is called a unary relationship. For example, one person is married to only one person.



Unary Relationship

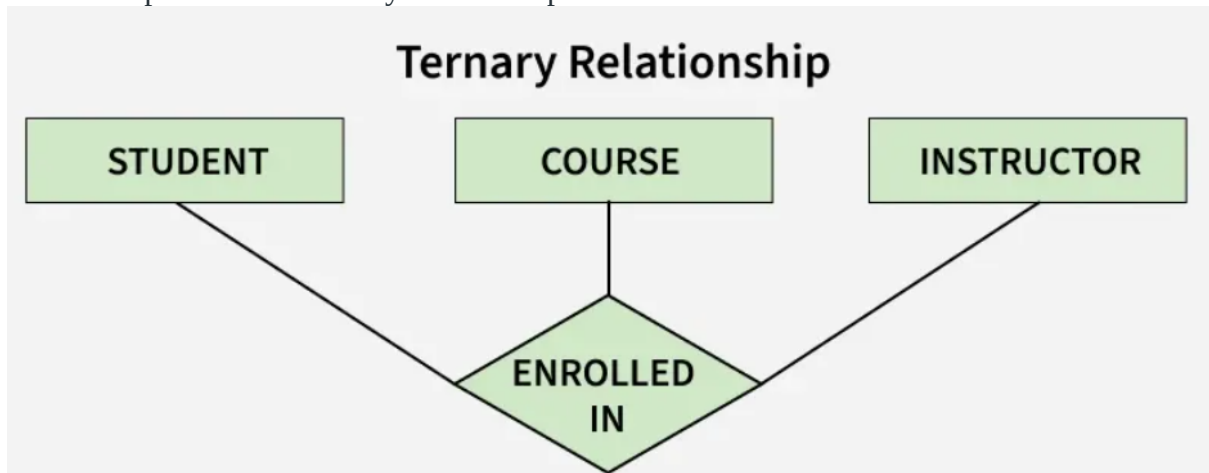
2. Binary Relationship: When there are TWO entities set participating in a relationship, the relationship is called a binary relationship. For example, a Student is enrolled in a Course.

i



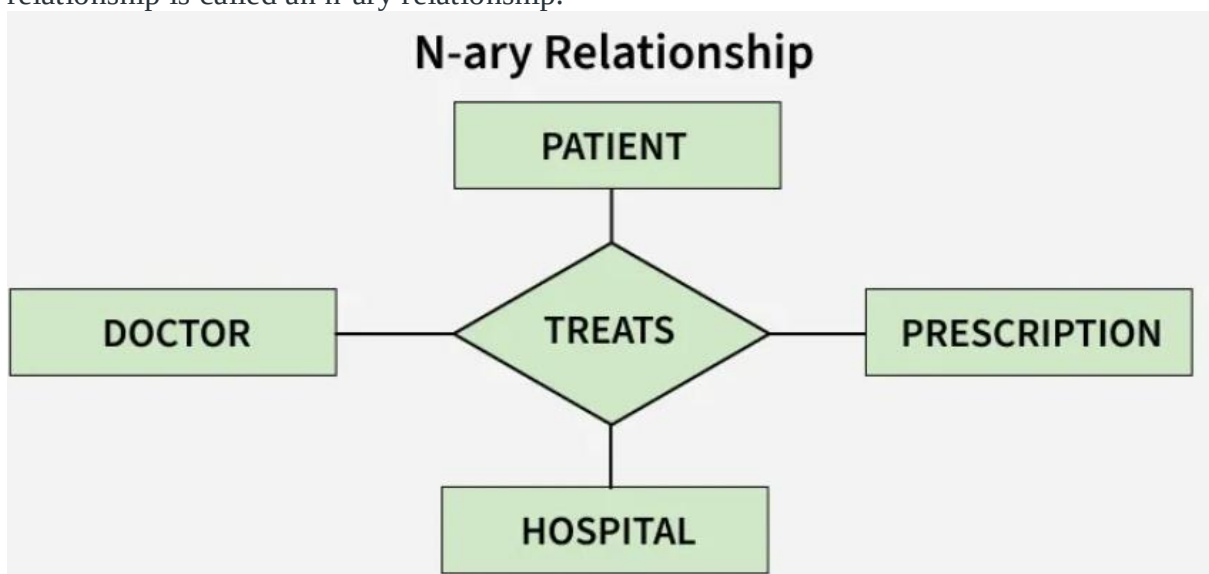
Binary Relationship

3. Ternary Relationship: When there are three entity sets participating in a relationship, the relationship is called a ternary relationship.



Ternary Relationship

4. N-ary Relationship: When there are n entities set participating in a relationship, the relationship is called an n-ary relationship.



N-ary Relationship

Cardinality in ER Model

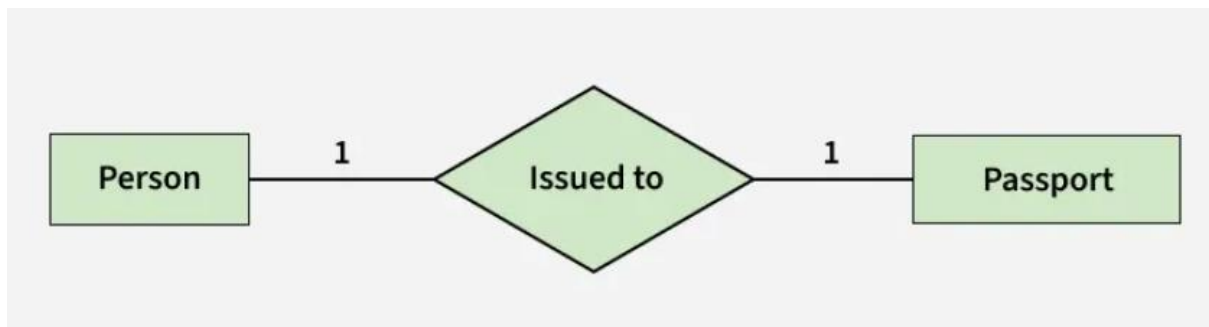
The maximum number of times an entity of an entity set participates in a relationship set is known as cardinality.

Cardinality can be of different types:

1. One-to-One

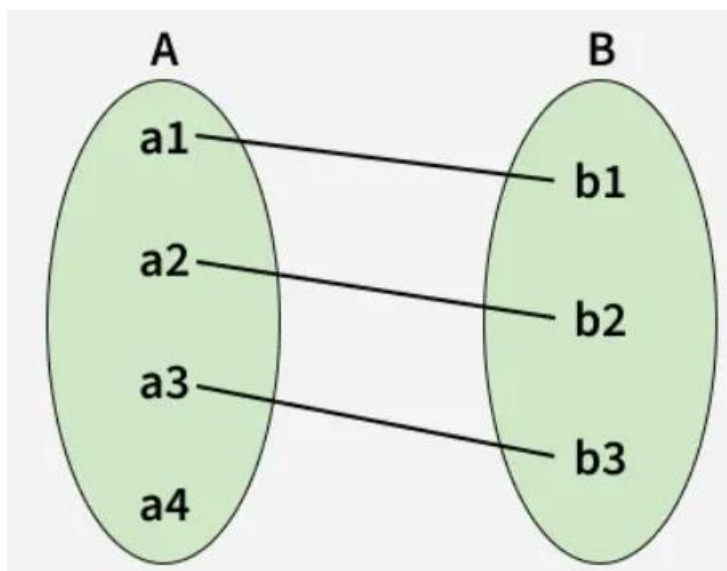
When each entity in each entity set can take part only once in the relationship, the cardinality is one-to-one. Let us assume that one person can be issued only one passport, and one

passport is issued to only one person. So, the relationship will be One-to-One (1 : 1), meaning that each person has a single passport, and each passport belongs to a single person.



one to one cardinality

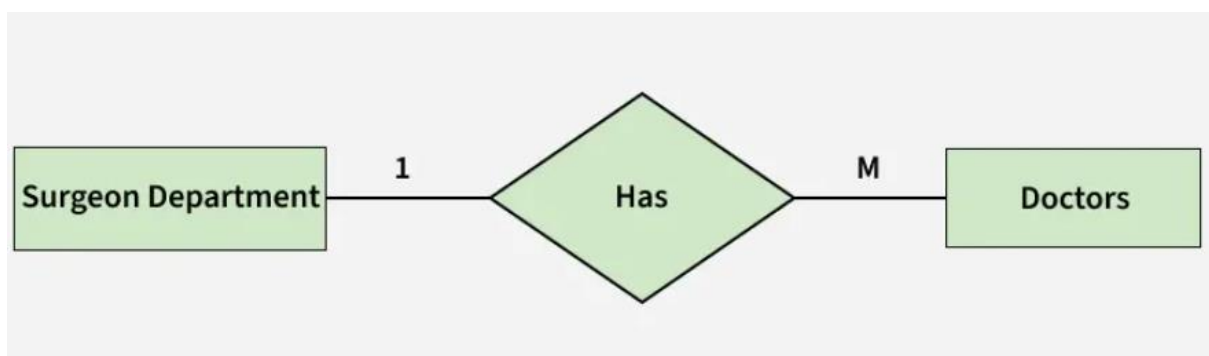
Using Sets, it can be represented as:



Set Representation of One-to-One

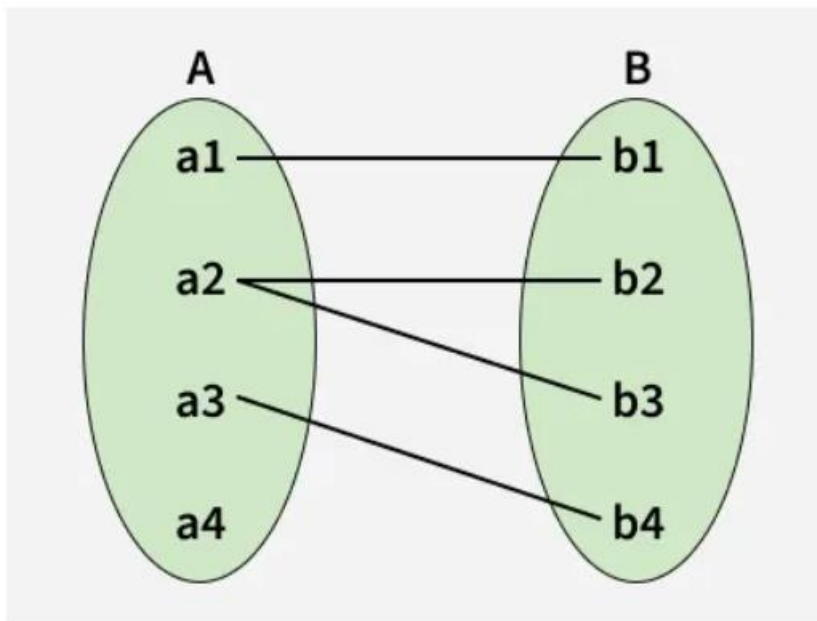
2. One-to-Many

In a one-to-many relationship, one entity can be associated with multiple entities. For example, a single Surgeon Department can have many Doctors. Therefore, the cardinality of this relationship is 1 to M, meaning one department can have many doctors.



one to many cardinality

Using sets, one-to-many cardinality can be represented as:

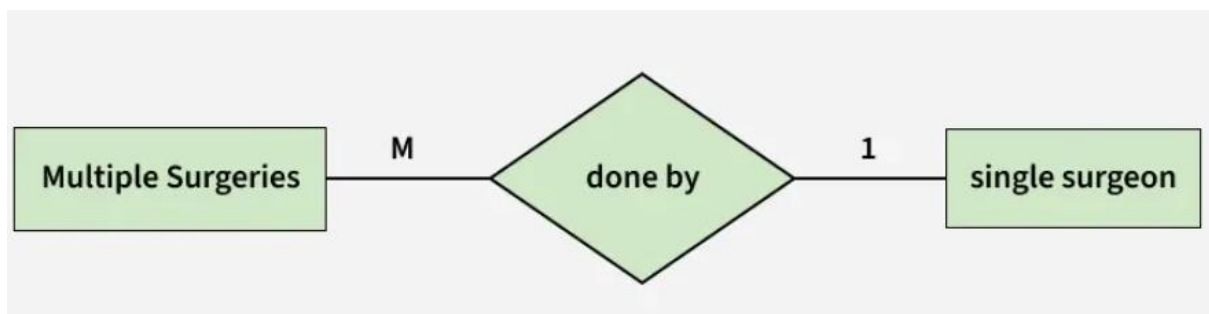


Set Representation of One-to-Many

3. Many-to-One

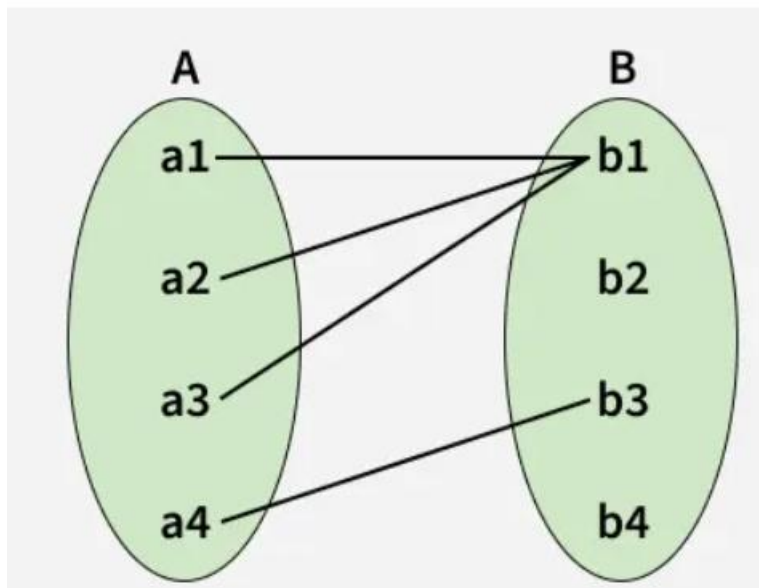
When entities in one entity set can take part only once in the relationship set and entities in other entity sets can take part more than once in the relationship set, cardinality is many to one.

Let us assume that multiple surgeries can be performed by one surgeon, but one surgery is performed by only one surgeon. So, the cardinality will be M to 1, meaning that many surgeries can be done by a single surgeon, but each surgery is done by only one surgeon.



many to one cardinality

Using Sets, it can be represented as:

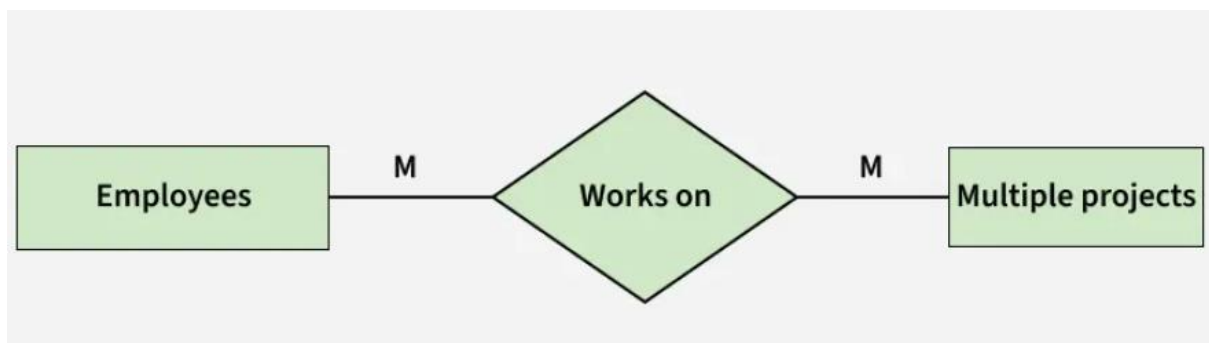


Set Representation of Many-to-One

In this case, each student is taking only 1 course but 1 course has been taken by many students.

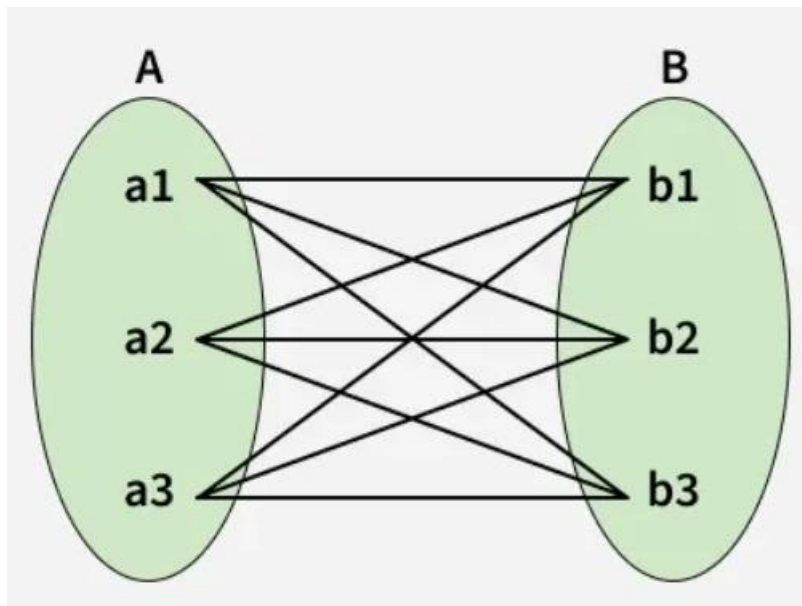
4. Many-to-Many

When entities in all entity sets can take part more than once in the relationship cardinality is many to many. Let us assume that an employee can work on multiple projects and each project can have multiple employees working on it. So, the relationship will be many-to-many (M:N), meaning that one employee may be associated with several projects, and one project may involve several employees.



many to many cardinality

Using Sets, it can be represented as:



Many-to-Many Set Representation

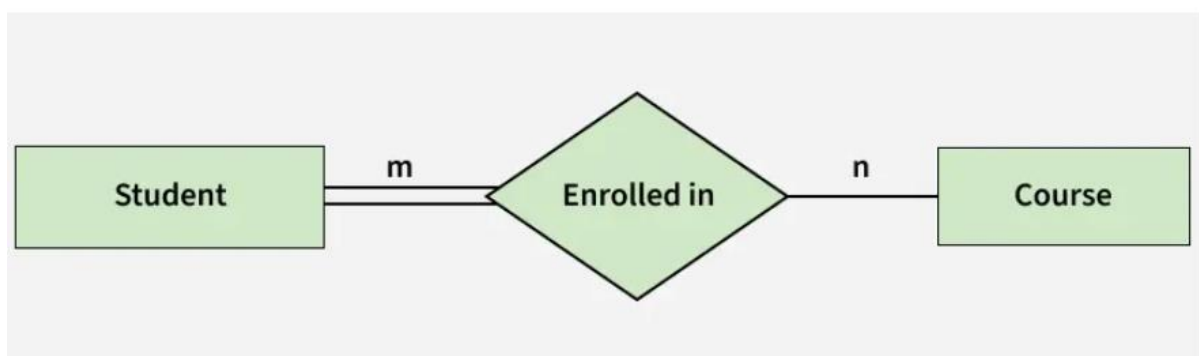
In this example, student A1 is enrolled in B1, B2 and B3, and course B3 is taken by A1, A2, and A3. Therefore, this represents a many-to-many relationship.

Participation Constraint

Participation Constraint is applied to the entity participating in the relationship set.

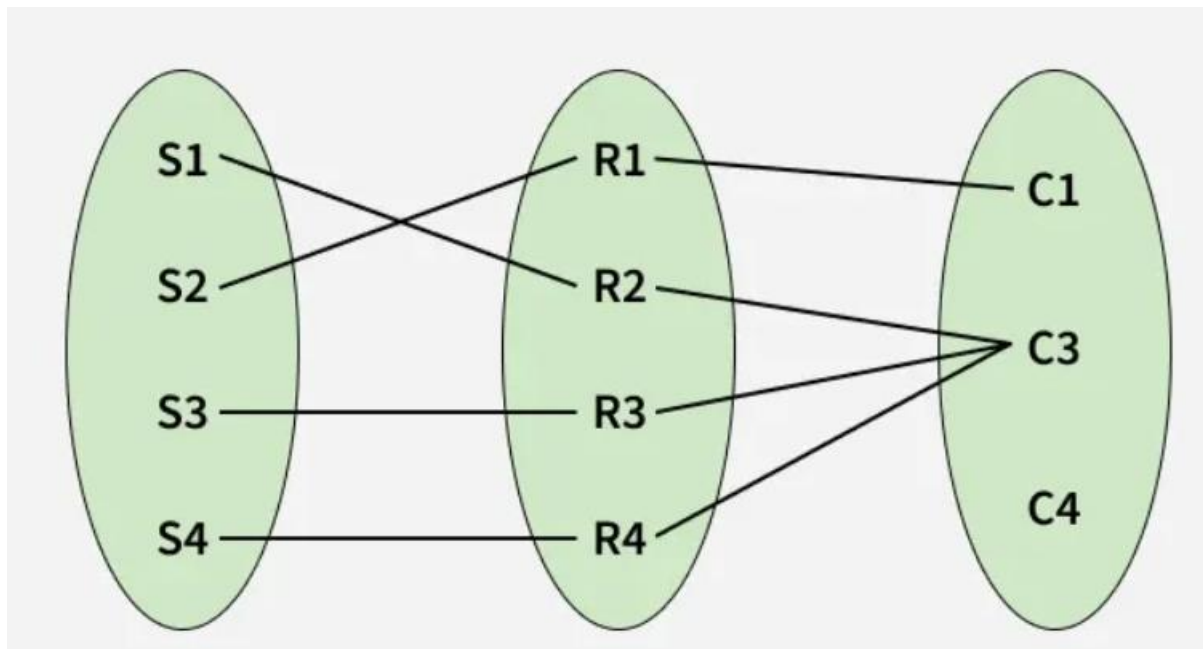
1. **Total Participation:** Each entity in the entity set must participate in the relationship. If each student must enroll in a course, the participation of students will be total. Total participation is shown by a double line in the ER diagram.
2. **Partial Participation:** The entity in the entity set may or may NOT participate in the relationship. If some courses are not enrolled by any of the students, the participation in the course will be partial.

The diagram depicts the 'Enrolled in' relationship set with Student Entity set having total participation and Course Entity set having partial participation.



Total Participation and Partial Participation

Using Set, it can be represented as,



Set representation of Total Participation and Partial Participation

Every student in the Student Entity set participates in a relationship but there exists a course C4 that is not taking part in the relationship.

How to Draw an ER Diagram

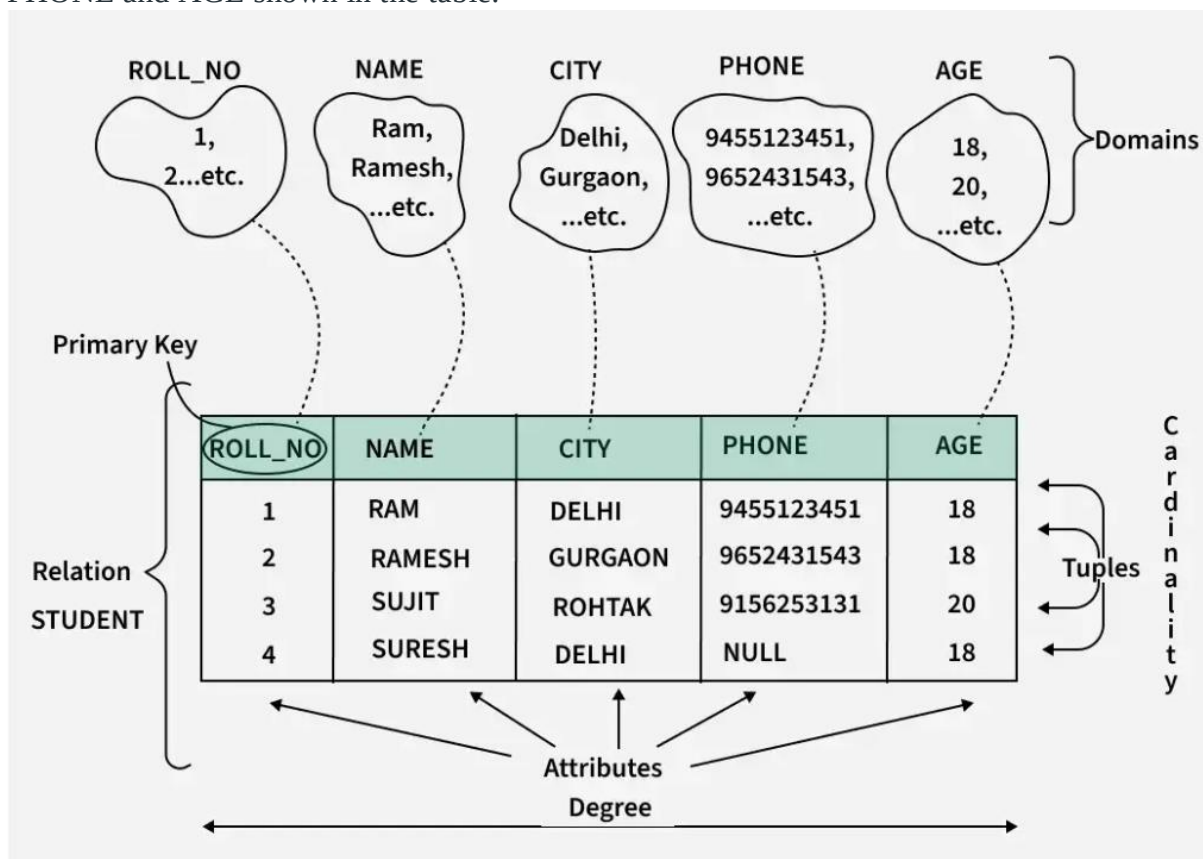
1. **Identify Entities:** The very first step is to identify all the Entities. Represent these entities in a Rectangle and label them accordingly.
2. **Identify Relationships:** The next step is to identify the relationship between them and represent them accordingly using the Diamond shape. Ensure that relationships are not directly connected to each other.
3. **Add Attributes:** Attach attributes to the entities by using ovals. Each entity can have multiple attributes (such as name, age, etc.), which are connected to the respective entity.
4. **Define Primary Keys:** Assign primary keys to each entity. These are unique identifiers that help distinguish each instance of the entity. Represent them with underlined attributes.
5. **Remove Redundancies:** Review the diagram and eliminate unnecessary or repetitive entities and relationships.
6. **Review for Clarity:** Review the diagram make sure it is clear and effectively conveys the relationships between the entities.

Relational Model in DBMS

The Relational Model organizes data using tables (relations) consisting of rows and columns.

- The relational model represents how data is stored and managed in Relational Databases where data is organized into tables, each known as a relation.
- Each row of a table represents an entity or record and each column represents a particular attribute of that entity.
- The relational model transforms conceptual designs from ER diagrams into implementable structures. These structures are used in relational database systems like Oracle SQL and MySQL.

Example: Consider a relation STUDENT with attributes ROLL_NO, NAME, ADDRESS, PHONE and AGE shown in the table.



Relational Model

Key Terms in the Relational Model

1. **Attribute:** Attributes are the properties that define an entity. For example, ROLL_NO, NAME, ADDRESS etc.
2. **Relation Schema:** A relation schema defines the structure of the relation and represents the name of the relation with its attributes. For example, STUDENT (ROLL_NO, NAME, ADDRESS, PHONE and AGE) is the relation schema for STUDENT. If a schema has more than 1 relation it is called Relational Schema.
3. **Tuple:** A Tuple represents a row in a relation. Each tuple contains a set of attribute values that describe a particular entity. For example, (1, RAM, DELHI, 9455123451, 18) is a tuple in the STUDENT table.
4. **Relation Instance:** The set of tuples of a relation at a particular instance of time is called a relation instance. It can change whenever there is an insertion, deletion or update in the database.
5. **Degree:** The number of attributes in the relation is known as the degree of the relation. For example, The STUDENT relation has a degree of 5, as it has 5 attributes.
6. **Cardinality:** The number of tuples in a relation is known as cardinality. For example, The STUDENT relation defined above has cardinality 4.
7. **NULL Values:** The value which is not known or unavailable is called a NULL value. It is represented by NULL. For example, PHONE of STUDENT having ROLL_NO 4 is NULL.

Types of Keys in the Relational Model

1. **Primary Key:** A Primary Key uniquely identifies each tuple in a relation. It must contain unique values and cannot have NULL values.
2. **Candidate Key:** A Candidate Key is a set of attributes that can uniquely identify a tuple in a relation.
3. **Super Key:** A Super Key is a set of attributes that can uniquely identify a tuple.
4. **Foreign Key:** A Foreign Key is an attribute in one relation that refers to the primary key of another relation.
5. **Composite Key:** A Composite Key is formed by combining two or more attributes to uniquely identify a tuple.

Relational Model Notation

- Relation schema R of degree n is denoted by $R(A_1, A_2, \dots, A_n)$.
- Uppercase letters Q, R, S denote relation names.
- Lowercase letters q, r, s denote relation states.
- Letters t, u, v denote tuples.
- In general, the name of a relation schema such as STUDENT also indicates the current set of tuples in that relation.
- An attribute A can be qualified with the relation name R to which it belongs by using the dot notation R.A for example, STUDENT.Name or STUDENT.Age.
- An n-tuple t in a relation r(R) is represented as $t = \langle v_1, v_2, \dots, v_n \rangle$ where v_i is the value corresponding to the attribute A_i . The value v_i for attribute A_i in tuple t can be accessed using $t[A_i]$ or $t.A_i$.

Characteristics of the Relational Model

- **Data Representation:** Data is organized in tables (relations), with rows (tuples) representing records and columns (attributes) representing data fields.
- **Atomic Values:** Each attribute in a table contains atomic values, meaning no multi-valued or nested data is allowed in a single cell.
- **Unique Keys:** Every table has a primary key to uniquely identify each record, ensuring no duplicate rows.
- **Attribute Domain:** Each attribute has a defined domain, specifying the valid data types and constraints for the values it can hold.
- **Data Independence:** The model ensures logical and physical data independence, allowing changes in the database schema without affecting the application layer.
- **Relational Operations:** Supports operations like selection, projection, join, union and intersection, enabling powerful data retrieval manipulation.
- **Data Consistency:** Ensures data consistency through constraints, reducing redundancy and anomalies.
- **Set-Based Representation:** Tables in the relational model are treated as sets and operations follow mathematical set theory principles.

Constraints in Relational Model

While designing the Relational Model, we define some conditions which must hold for data present in the database are called Constraints. These constraints are checked before performing any operation (insertion, deletion and updation) in the database. If there is a violation of any of the constraints, the operation will fail.

1. Domain Constraints

Domain Constraints ensure that the value of each attribute A in a tuple must be an atomic value derived from its specified domain, $\text{dom}(A)$. Domains are defined by the data types associated with the attributes. Common data types include:

- **Numeric types:** Includes integers (short, regular and long) for whole numbers and real numbers (float, double-precision) for decimal values, allowing precise calculations.
- **Character types:** Consists of fixed-length (CHAR) and variable-length (VARCHAR, TEXT) strings for storing text data of various sizes.
- **Boolean values:** Stores true or false values, often used for flags or conditional checks in databases.
- **Specialized types:** Includes types for date (DATE), time (TIME), timestamp (TIMESTAMP) and money (MONEY), used for precise handling of time-related and financial data.

2. Key Integrity

Every relation in the database should have at least one set of attributes that defines a tuple uniquely. Those set of attributes is called keys. e.g.; ROLL_NO in STUDENT is key. No two students can have the same roll number. So a key has two properties:

- It should be unique for all tuples.
- It can't have NULL values.

3. Referential Integrity Constraints

When one attribute of a relation can only take values from another attribute of the same relation or any other relation, it is called [referential integrity](#). Let us suppose we have 2 relations

Table: STUDENT

ROLL_NO	NAME	ADDRESS	PHONE	AGE	BRANCH_CODE
1	RAM	DELHI	9455123451	18	CS
2	RAMESH	GURGAON	9652431543	18	CS
3	SUJIT	ROHTAK	9156253131	20	ECE
4	SURESH	DELHI		18	IT

Table: BRANCH

BRANCH_CODE	BRANCH_NAME
CS	COMPUTER SCIENCE
IT	INFORMATION TECHNOLOGY
ECE	ELECTRONICS AND COMMUNICATION ENGINEERING
CV	CIVIL ENGINEERING

Explanation: BRANCH_CODE of STUDENT can only take the values which are present in BRANCH_CODE of BRANCH which is called referential integrity constraint. The relation which is referencing another relation is called REFERENCING RELATION (STUDENT in this case) and the relation to which other relations refer is called REFERENCED RELATION (BRANCH in this case).

Anomalies in the Relational Model

An anomaly is an irregularity or something which deviates from the expected or normal state. When designing databases, we identify three types of anomalies: Insert, Update, and Delete.

- **Insertion Anomaly in Referencing Relation:** We can't insert a row in REFERENCING RELATION if referencing attribute's value is not present in the referenced attribute value. e.g.; Insertion of a student with BRANCH_CODE 'ME' in STUDENT relation will result in an error because 'ME' is not present in BRANCH_CODE of BRANCH.
- **Deletion/ Updation Anomaly in Referenced Relation:** We can't delete or update a row from REFERENCED RELATION if the value of REFERENCED ATTRIBUTE is used in the value of REFERENCING ATTRIBUTE. For example, If we try to delete a tuple from BRANCH having BRANCH_CODE 'CS', it will result in an error because 'CS' is referenced by BRANCH_CODE of STUDENT, but if we try to delete the row from BRANCH with BRANCH_CODE CV, it will be deleted as the value is not been used by referencing relation.

Codd Rules in Relational Model

E.F. Codd, the creator of the relational model, proposed 12 rules (known as Codd's 12 Rules) that define what constitutes a relational database system. These rules emphasize the importance of data independence, consistency and structure.

Key Codd's Rules

- **Rule 1:** The information rule : All information in a relational database is represented logically in tables (relations).
- **Rule 2:** The guaranteed access rule : Every data element is accessible by using a combination of table name, primary key and attribute name.

- **Rule 5:** The powerful language rule : A relational DBMS should have a comprehensive language capable of expressing all relational queries.

Advantages of the Relational Model

- **Simple model:** Relational Model is simple and easy to use in comparison to other languages.
- **Flexible:** Relational Model is more flexible than any other relational model present.
- **Secure:** Relational Model is more secure than any other relational model.
- **Data Accuracy:** Data is more accurate in the relational data model.
- **Data Integrity:** The integrity of the data is maintained in the relational model.
- **Operations can be Applied Easily:** It is better to perform operations in the relational model.

Disadvantages of the Relational Model

- **Performance:** The relational model can experience performance issues with very large databases.
- **Complexity for Complex Data:** The model struggles with hierarchical or complex data relationships, which might be better handled with other models like the Graph or Document model.
- **Normalization Overhead:** Extensive use of normalization can result in complex queries and slower performance.

Database Languages

Database languages are specialized sets of commands and instructions used to define, manipulate and control data within a database. Each language type plays a distinct role in database management, ensuring efficient storage, retrieval and security of data. The primary database languages include:

Type of DBMS Language				
Data Definition Language	Data Manipulation Language	Data Control Language	Transaction Control Language	Data Query Language
• CREATE • DROP • TRUNCATE • RENAME • ALTER	• INSERT • DELETE • UPDATE	• REVOKE • GRANT	• ROLLBACK • COMMIT	• SELECT

1. Data Definition Language (DDL)

DDL is the short name for Data Definition Language, which deals with database schemas and descriptions, of how the data should reside in the database.

- **CREATE:** to create a database and its objects like (table, index, views, store procedure, function and triggers)
- **ALTER:** alters the structure of the existing database
- **DROP:** delete objects from the database
- **TRUNCATE:** remove all records from a table, including all spaces allocated for the records are removed

2. Data Manipulation Language (DML)

DML focuses on manipulating the data stored in the database, enabling users to retrieve, add, update and delete data.

- **INSERT:** insert data into a table
- **UPDATE:** updates existing data within a table
- **DELETE:** Delete all records from a database table

3. Data Control Language (DCL)

DCL commands manage access permissions, ensuring data security by controlling who can perform certain actions on the database.

- **GRANT:** Provides specific privileges to a user (e.g., SELECT, INSERT).
- **REVOKE:** Removes previously granted permissions from a user.

4. Transaction Control Language (TCL)

TCL commands oversee transactional data to maintain consistency, reliability and atomicity.

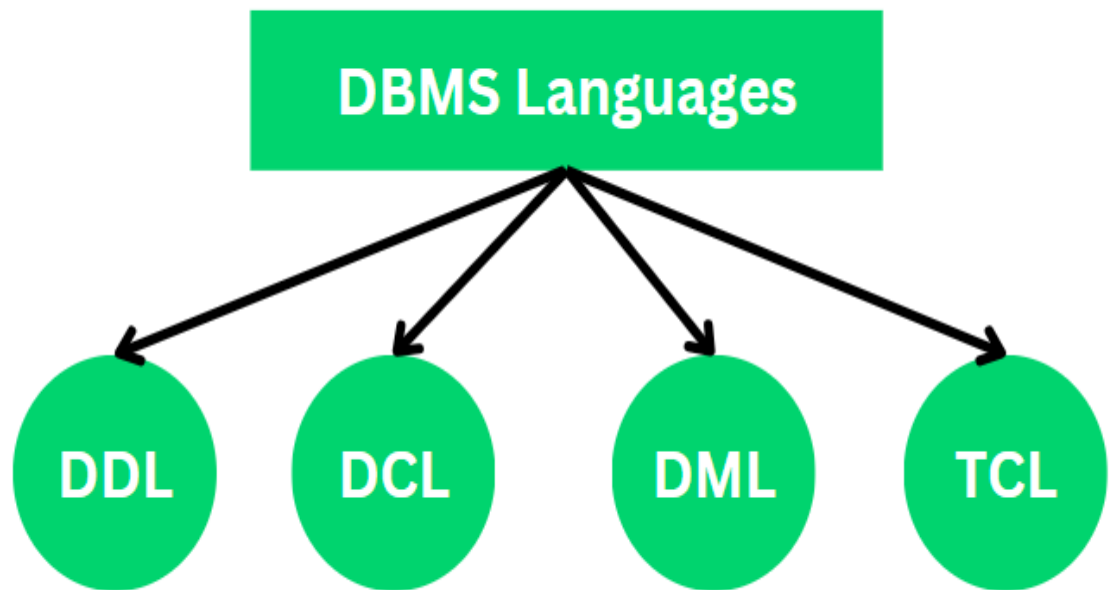
- **ROLLBACK:** Undoes changes made during a transaction.
- **COMMIT:** Saves all changes made during a transaction.
- **SAVEPOINT:** Sets a point within a transaction to which one can later roll back.

5. Data Query Language (DQL)

DQL is a subset of SQL used to retrieve data from a database without modifying it. Its main command is SELECT, which allows users to fetch specific information based on their requirements.

Database Languages in DBMS

Database languages are specialized languages used to interact with a database. They allow users to perform different tasks such as defining, controlling and manipulating the data. There are several types of database languages in DBMS, categorized into the following four main types:



Database Languages

1. DDL (Data Definition Language)
2. DCL (Data Control Language)
3. DML (Data Manipulation Language)
4. TCL (Transaction Control Language)

1. DDL (Data Definition Language)

The DDL stands for Data Definition Language, Which is used to define the database's internal structure and Pattern of the Database. It is used to define and modify the structure of the database itself, including the tables, views, indexes and other schema-related objects. It deals with the creation and modification of database schema, but it doesn't deal with the data itself. Following are the five DDL commands in SQL:

- **CREATE:** Used to create database objects like tables, indexes or views.
- **ALTER:** Used to modify the structure of an existing database object, such as adding a new column to a table.
- **DROP:** Used to delete database objects.
- **TRUNCATE:** Used to remove all rows from a table, without affecting the structure.
- **RENAME:** Used to change the name of a database object.

1.1 CREATE Command

The CREATE is a DDL command used to create databases, tables, triggers and other database objects.

Syntax:

```
CREATE TABLE Students (  
column1 INT,  
column2 VARCHAR(50),  
column3 INT  
);
```

Example:

```
mysql> CREATE TABLE Students (  
->     StudentID INT PRIMARY KEY,  
->     Name VARCHAR(50),  
->     Age INT  
-> );
```

Query OK, 0 rows affected (0.05 sec)

```
mysql> desc students;
```

Field	Type	Null	Key	Default	Extra
StudentID	int	NO	PRI	NULL	
Name	varchar(50)	YES		NULL	
Age	int	YES		NULL	

3 rows in set (0.01 sec)

1.2 Alter Command

ALTER is a DDL command which changes or modifies the existing structure of the database and it also changes the schema of database objects. We can also add and drop constraints of the table using the ALTER command.

Syntax:

```
ALTER TABLE Students ADD column_name;
```

Example:

```
mysql> ALTER TABLE Students
-> ADD Weight DECIMAL(5,2);
Query OK, 0 rows affected (0.06 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> desc students;
```

Field	Type	Null	Key	Default	Extra
StudentID	int	NO	PRI	NULL	
Name	varchar(50)	YES		NULL	
Age	int	YES		NULL	
Weight	decimal(5,2)	YES		NULL	

4 rows in set (0.00 sec)

1.3 Drop Command

DROP is a DDL command used to delete/remove the database objects from the SQL database. We can easily remove the entire table, view or index from the database using this DDL command.

Syntax:

DROP Table Table_name;

Example:

```
mysql> drop table students;
Query OK, 0 rows affected (0.05 sec)
```

1.4 Truncate Command

The TRUNCATE command is used to delete all the records from a table without removing the structure. Unlike the DELETE command, which can remove specific rows and can be rolled back, TRUNCATE is a more efficient way to delete all rows in a table without logging individual row deletions.

Syntax:

TRUNCATE TABLE table_name;

Example:

```
mysql> select * from students;
+-----+-----+-----+-----+
| StudentID | Name      | Age  | Weight |
+-----+-----+-----+-----+
|          1 | John Doe |   20 |  65.50 |
+-----+-----+-----+-----+
1 row in set (0.02 sec)

mysql> TRUNCATE TABLE Students;
Query OK, 0 rows affected (0.04 sec)

mysql> select * from students;
Empty set (0.00 sec)
```

1.5 Rename Command

The RENAME command in a Database Management System (DBMS) is used to change the name of a database object, such as a table, column or index. This command is helpful when we want to give a more meaningful or appropriate name to an object without needing to recreate it.

Syntax:

```
ALTER TABLE Old_Table_Name RENAME TO New_Table_Name;
```

Example:

```
mysql> ALTER TABLE Students
-> RENAME TO ClassMembers;
Query OK, 0 rows affected (0.02 sec)
```

2. DCL (Data Control Language)

DCL stands for Data Control Language. It is used to control the access permissions of users to the database. DCL commands help grant or revoke privileges to users, determining who can perform actions like reading or modifying data. DCL commands are transactional, meaning they can be rolled back if necessary. The two main DCL commands are:

- **Grant:** Gives user access to the database
- **Revoke:** Removes access or permissions from the user

2.1 Grant Command

The GRANT command in a Database Management System (DBMS) is used to provide specific permissions or privileges to users or roles. This command allows administrators to control access to database objects, ensuring that only authorized users can perform certain actions, such as selecting, inserting, updating or deleting data.

Syntax:
GRANT privileges
ON object
TO user_or_role [WITH GRANT OPTION];
Example:
GRANT SELECT, INSERT ON students TO user;

2.2 Revoke Command

The REVOKE command in a Database Management System (DBMS) is used to remove previously granted permissions or privileges from users or roles. This command is essential for managing access control, ensuring that users do not have more privileges than necessary to perform their tasks.

Syntax:
REVOKE privileges ON object FROM user_or_role;
Example:
REVOKE ALL PRIVILEGES ON students FROM user;

3. DML (Data Manipulation Language)

The DML (Data Manipulation Language) is used to manage and manipulate data within a database. With DML, you can perform various operations such as inserting, updating, selecting and deleting data. These operations allow you to work with the actual content in your database tables. Here are the key DML commands:

- **SELECT:** Retrieves data from the table based on specific criteria.
- **INSERT:** Adds new rows of data into an existing table.
- **UPDATE:** Modifies existing data in a table.
- **DELETE:** Removes data from a table.

3.1 SELECT Command

The SELECT command in SQL (Structured Query Language) is used to retrieve data from one or more tables in a database. It is the most commonly used commands in SQL, allowing users to specify which columns and rows of data they want to retrieve and how they want that data organized. The SELECT statement can be used in various ways, such as selecting all data from a table, filtering records based on conditions or sorting the results.

Syntax:
SELECT * FROM Table_Name

Example:

```
mysql> select * from ClassMembers;
+-----+-----+-----+-----+
| StudentID | Name      | Age  | Weight |
+-----+-----+-----+-----+
|          1 | John Doe  | 20   | 65.50  |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

3.2 Insert Command

The INSERT command in SQL (Structured Query Language) is used to add new records or rows to a table in a database. It is a key operation in database management, essential for populating tables with new data. This command can insert data into all columns or specific columns of a table.

Syntax:

INSERT INTO Table_Name (Column 1, Column 2, Column 3, Column 4) VALUES (Value 1, Value 2, Value 3, Value 4);

Example:

```
mysql> INSERT INTO ClassMembers (StudentID, Name, Age, Weight)
-> VALUES (2, 'Alice Smith', 22, 60.2);
Query OK, 1 row affected (0.03 sec)

mysql> select * from ClassMembers;
+-----+-----+-----+-----+
| StudentID | Name      | Age  | Weight |
+-----+-----+-----+-----+
|          1 | John Doe  | 20   | 65.50  |
|          2 | Alice Smith | 22   | 60.20  |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

3.3 Update Command

The UPDATE command in SQL (Structured Query Language) is used to modify existing records in a table. This command enables users to change the values of one or more columns for specific rows based on a condition (criteria). It is important for maintaining and adjusting data in a database when necessary.

Syntax:

UPDATE Table_Name SET Name = 'New_Value' WHERE Name = 'Old_Value';

Example:

```
mysql> UPDATE ClassMembers
      -> SET Name = 'Roman'
      -> WHERE Name = 'John Doe';
Query OK, 1 row affected (0.03 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from ClassMembers;
+-----+-----+-----+-----+
| StudentID | Name      | Age  | Weight |
+-----+-----+-----+-----+
|          1 | Roman     | 20   | 65.50  |
|          2 | Alice Smith | 22   | 60.20  |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

3.4 Delete Command

The DELETE command in SQL (Structured Query Language) is used to remove one or more existing records from a table in a database. It is an essential operation for managing data, enabling users to delete specific rows that meet certain conditions or criteria.

Syntax:

DELETE FROM Table_Name WHERE Column = Value;

Example:

```
mysql> DELETE FROM ClassMembers
      -> WHERE StudentID = 2;
Query OK, 1 row affected (0.03 sec)

mysql> select * from ClassMembers;
+-----+-----+-----+-----+
| StudentID | Name  | Age  | Weight |
+-----+-----+-----+-----+
|          1 | Roman | 20   | 65.50  |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

4. TCL (Transaction Control Language)

The TCL full form is Transaction Control Language commands are used to manage and control transactions in a database, grouping them into logical units. These commands help ensure the integrity of data and consistency during complex operations. Here are the two main commands in this category:

- **Commit:** Saves all the changes made during the current transaction to the database. These are very useful in the banking sector.
- **Rollback:** used to restore the database to its original state from the last commit. This command also plays an important role in Banking Sectors.

4.1 Commit Command

The COMMIT command is used to save all changes made during a transaction in the database. This command ensures that modifications made by DML statements (such as INSERT, UPDATE or DELETE) become permanent in the database.

Syntax:

Database Operation
Commit

Example:

UPDATE Student SET Marks = 90 WHERE RollNo = 101;

COMMIT;

4.2 ROLLBACK Command

The rollback command is used to restore the database to its state at the last COMMIT, effectively undoing any changes made since that point. It helps ensure data consistency by allowing the reversal of partial or erroneous operations.

Syntax:

ROLLBACK;

Example: DELETE FROM Student WHERE RollNo = 101;

ROLLBACK;

3.SAVEPOINT

Definition:SAVEPOINT is used to create a point inside a **transaction** to which we can rollback later.

Example: INSERT INTO Student VALUES (101, 'Riya', 85);

SAVEPOINT A;

UPDATE Student SET Marks = 95 WHERE RollNo = 101;

ROLLBACK TO A;